Phase 1: Data & Infrastructure

Goal: Get raw data flowing and the database ready. You cannot build a model or a UI without data.

1. **Setup Environment:**
    - Initialize backend/ with requirements.txt.
    - Create .env and backend/app/core/config.py.
2. **Database & Schema:**
    - Define the schema in backend/app/db/schema.sql. You need tables for: market_data, users (AI vs Human), trades, portfolio_snapshot, and leaderboard.
    - Implement backend/app/db/db.py to ensure you can connect.
3. **Data Ingestion (Crucial First Step):**
    - Build backend/app/pipelines/ingestion.py.
    - **Task:** Write the script to fetch 5 years of historical OHLCV data and save it to backend/data/raw/.
    - **Why:** You need this static dataset to train models and run backtests later.

Phase 2: The Brain (ML & Strategy Logic)

Goal: Create the AI agent and the rules of the game.

1. **The "Constitution" (Contracts):**
    - Implement backend/app/core/contracts.py. Define the rules: Max drawdown, starting capital, allowed leverage, and trading hours.
    - **Why:** Both the AI and Human modules will import this. Doing it now prevents refactoring later.
2. **Feature Engineering:**
    - Build backend/app/pipelines/features.py and sentiment.py.
    - Transform your raw data into training data (RSI, MACD, FinBERT scores). Save this to backend/data/features/.
3. **Model Training (MVP):**

- Work in backend/models/train.py.
- Train a simple model (e.g., Random Forest or XGBoost) and save the artifact as backend/models/model.pkl.
- Implement backend/models/predict.py to ensure you can load the pickle and generate a signal from new data.

Phase 3: The Engine (Backend Business Logic)

Goal: Turn the model and data into actionable trade logic.

1. **Execution & Position Management:**
   - Implement backend/app/strategy/position_sizer.py and risk_manager.py.
   - Build backend/app/execution/execution_engine.py. This needs to take a "Signal" (Buy/Sell) and update the portfolio database table.
2. **The AI Player:**
   - Build backend/app/api/ai.py. This endpoint should trigger the pipeline: *Ingest -> Feature -> Predict -> Risk Check -> Execute*.
3. **The Human Player:**
   - Build backend/app/human/validator.py (checks against contracts.py).
   - Build backend/app/api/human.py to accept manual trade entries.
4. **Explainability:**
   - Implement backend/app/explainability/shap_explainer.py.
   - Ensure the AI generates a JSON "reasoning" object whenever it trades.

Phase 4: The Interface (API Layer)

Goal: Expose everything via HTTP so the frontend can read it.

1. **FastAPI Setup:**
   - Finalize backend/app/main.py and backend/app/api/routes.py.
2. **Endpoints:**

- **Market Data:** GET /market/history (for the frontend charts).
- **Portfolio:** GET /portfolio/stats (Equity, PnL).
- **Leaderboard:** GET /leaderboard (Compares AI vs Human equity).

3. **Simulation Script:**
   - Write backend/scripts/run_simulation.py.
   - This script should simulate "Time passing." It updates the market data, triggers the AI to trade, and updates the leaderboard at the end of every "day."

Phase 5: The Face (Frontend)

Goal: Visualize the data. Do this last so you don't have to mock data.

1. **Setup Next.js:**
   - Initialize frontend/ and configure tailwind.config.ts.
   - Set up frontend/lib/api.ts to fetch from your running FastAPI server.
2. **Components (Bottom-Up):**
   - Build EquityCurve.tsx and TradeChart.tsx first (using a charting library like Recharts or Lightweight Charts).
   - Build TradeExplanation.tsx to render the SHAP values/text.
3. **Pages:**
   - **Dashboard (page.tsx):** Assemble the charts.
   - **Human Interface (human/page.tsx):** Create the Buy/Sell buttons that call your human.py API.
   - **Leaderboard (leaderboard/page.tsx):** A simple table comparing the two portfolios.

Summary of the Critical Path

1. **Data** (Ingest) —> 2. **Logic** (Train/Rules) —> 3. **API** (Endpoints) —> 4. **UI** (Visuals).